docker

Docker pour Python et FastAPI **1**
Docker 1
Qu'est-ce que Docker ? 1
Installation et premiers pas **2**
Installer Docker 2
Installer Docker Compose 2
Créer un premier conteneur **3**
Dockerfile simple pour un script Python 3
Build et run 3
Lancer le conteneur : 3
Docker pour FastAPI **4**
Dockerfile pour une API FastAPI 4
Exposer des ports 5
Docker Compose pour gérer plusieurs services **5**
Introduction à docker-compose.yml 5
Exemple API + base de données PostgreSQL 6
Optimisations pour production 7
Images légères 7
Variables d'environnement et secrets 7

Docker pour Python et FastAPI

Docker

Docker est une technologie qui permet d'exécuter des applications dans des environnements isolés appelés conteneurs.
Pour un développeur Python, Docker présente plusieurs avantages :

- Déployer des applications sans se soucier de l'environnement
- Tester en local un environnement proche de la production
- Simplifier le déploiement sur le cloud ou chez un client

En résumé :

Docker = "ça marche chez moi" -> "ça marche partout"

Qu'est-ce que Docker ?

Docker utilise des images (modèles d'applications) pour créer des conteneurs (instances isolées). Ces conteneurs exécutent votre code, vos bibliothèques et leurs dépendances sans interagir avec le système hôte.

Exemple :

- Vous développez une API FastAPI avec Python 3.12
- Le serveur distant n'a que Python 3.8 ? Pas grave : votre conteneur aura Python 3.12

Pourquoi utiliser Docker pour Python et FastAPI ?

- Standardisation : votre API fonctionnera pareil en local, sur serveur, dans le cloud
- Isolation : aucun risque de conflit entre projets ou dépendances
- Déploiement facile : tout est automatisable (CI/CD, tests, livraison)

- Scalabilité : très facile de "scaler" votre API en créant plusieurs conteneurs

Installation et premiers pas

Installer Docker

Sur Linux/Mac/Windows, vous pouvez installer Docker Desktop depuis : https://www.docker.com/products/docker-desktop

Vérifiez ensuite que Docker est bien installé :

```
docker --version
```

Test rapide :

```
docker run hello-world
```

Cela lance un mini conteneur qui affiche un message si tout fonctionne.

```
Hello from Docker!
```

Installer Docker Compose

Docker Compose est un outil pour gérer plusieurs conteneurs ensemble.

Sur Docker Desktop, il est déjà inclus. Sinon, installez-le :

```
sudo apt install docker-compose
```

Vérification :

```
docker-compose --version
```

Créer un premier conteneur

Dockerfile simple pour un script Python

Créez un fichier Dockerfile :

```
# Image officielle Python
FROM python:3.12-slim

# Dossier de travail
WORKDIR /app

# Copier le script
COPY app.py .

# Commande à exécuter
CMD ["python", "app.py"]
```

Et un petit app.py :

```
print("Hello depuis Docker !")
```

Build et run

Build de l'image (attention le "." est très important) :

```
docker build -t my-python-app .
```

Lancer le conteneur :

```
docker run my-python-app
```

Résultat : Hello depuis Docker !

Docker pour FastAPI

Dockerfile pour une API FastAPI

Pour une API FastAPI, on a besoin également d'installer fastapi et uvicorn.

requirements.txt :

```
fastapi
uvicorn[standard]
```

Voici un Dockerfile adapté :

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello depuis FastAPI et Docker"}
```

Exposer des ports

Quand on lance un conteneur, il faut exposer les ports vers l'extérieur :

```
docker build -t my-fastapi-app .
docker run -p 8000:8000 my-fastapi-app
```

Naviguez ensuite sur http://localhost:8000

Docker Compose pour gérer plusieurs services

Introduction à docker-compose.yml

Quand une application à plusieurs composants (ex: API + base de données), Docker Compose simplifie tout.

Exemple docker-compose.yml minimal :

```
version: '3'

services:
  api:
    build: .
    ports:
      - "8000:8000"
```

Commande pour lancer :

```
docker-compose up
```

Exemple API + base de données PostgreSQL

```yaml
version: '3'

services:
  db:
    image: postgres
    restart: always
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"

  api:
    build: ./api
    ports:
      - "8000:8000"
    depends_on:
      - db
```

Votre API pourra ensuite se connecter à PostgreSQL via l'url db:5432

Optimisations pour production

Images légères

Utilisez des versions "slim" ou Alpine pour avoir des conteneurs petits et rapides.

```
FROM python:3.12-slim
```

Variables d'environnement et secrets

Ne mettez jamais de mots de passe directement dans le code. Utilisez des variables d'environnement :